Semester	B.E. Semester VII – CMPN
Subject	R Programming LAB
Subject Professor In-charge	Prof. Sanjeev Dwivedi
Assisting Teachers	
Laboratory	M411

Student Name	Eshanika Amballa	
Roll Number	23102B0056	
Grade and Subject Teacher's Signature		

Experiment Number	08
Experiment Title	**High-Performance Big Data Analytics Using R**
Process	**Problem statement** The NYC Taxi and Limousine Commission records millions of taxi trips. Normal ways of processing this much data in R can be slow and use a lot of memory. In this lab I analyse the yellow taxi data with data.table, compare it with functional (apply, lapply, purrr) and vectorized code, run one heavy job sequentially and in parallel with foreach and doParallel, and benchmark everything to decide which approach is best for large data. **Dataset** NYC TLC Yellow Taxi Trip Records for January, February and March 2023 (about 9.4 million raw trips) and the Taxi Zone Lookup table. Source: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page Columns used: pickup and dropoff time, pickup and dropoff location ID, passenger count, trip distance, payment type, fare, tip, tolls and total amount. The files are downloaded inside the notebook, so nothing has to be uploaded. If the download is blocked, download the three parquet files and the zone csv from the TLC page and upload them to the Colab session with the same file names.

```r
# structure, size and summary of the raw data
dim(trips)
str(trips)
summary(trips)
sapply(trips, function(x) sum(is.na(x)))     # missing values per column
print(object.size(trips), units = "MB")
dim(zones)
head(zones)
```

```
9384487 11
Classes ‘data.table’ and 'data.frame':  9384487 obs. of  11 variables:
 $ tpep_pickup_datetime : POSIXct, format: "2023-01-01 00:32:10" "2023-01-01 00:55:08" ...
 $ tpep_dropoff_datetime: POSIXct, format: "2023-01-01 00:40:36" "2023-01-01 01:01:27" ...
 $ passenger_count      : num  1 1 1 0 1 1 1 1 1 1 ...
 $ trip_distance        : num  0.97 1.1 2.51 1.9 1.43 1.84 1.66 11.7 2.95 3.01 ...
 $ PULocationID         : num  161 43 48 138 107 161 239 142 164 141 ...
 $ DOLocationID         : num  141 237 238 7 79 137 143 200 236 107 ...
 $ payment_type         : num  2 1 1 1 1 1 1 1 1 2 ...
 $ fare_amount          : num  9.3 7.9 14.9 12.1 11.4 12.8 12.1 45.7 17.7 14.9 ...
 $ tip_amount           : num  0 4 15 0 3.28 ...
 $ tolls_amount         : num  0 0 0 0 0 0 0 3 0 0 ...
 $ total_amount         : num  14.3 16.9 34.9 20.9 19.7 ...
 - attr(*, ".internal.selfref")=<pointer: 0x5a47ad1ad3f0>
 tpep_pickup_datetime          tpep_dropoff_datetime         passenger_count
 Min.   :2001-01-01 00:06:49   Min.   :2001-01-01 14:13:51   Min.   :0.000
 1st Qu.:2023-01-24 23:33:22   1st Qu.:2023-01-24 23:48:47   1st Qu.:1.000
 Median :2023-02-16 14:58:37   Median :2023-02-16 15:17:08   Median :1.000
 Mean   :2023-02-16 05:29:55   Mean   :2023-02-16 05:46:08   Mean   :1.355
 3rd Qu.:2023-03-10 14:34:35   3rd Qu.:2023-03-10 14:54:20   3rd Qu.:1.000
 Max.   :2023-04-05 20:17:42   Max.   :2023-04-05 20:35:28   Max.   :9.000
                                                             NAs    :236179
 trip_distance       PULocationID  DOLocationID    payment_type
 Min.   :0.00e+00   Min.   :  1   Min.   :  1.0   Min.   :0.000
 1st Qu.:1.06e+00   1st Qu.:132   1st Qu.:114.0   1st Qu.:1.000
 Median :1.79e+00   Median :162   Median :162.0   Median :1.000
 Mean   :3.87e+00   Mean   :166   Mean   :164.2   Mean   :1.188
 3rd Qu.:3.33e+00   3rd Qu.:234   3rd Qu.:234.0   3rd Qu.:1.000
 Max.   :3.35e+05   Max.   :265   Max.   :265.0   Max.   :5.000

  fare_amount        tip_amount       tolls_amount       total_amount
 Min.   :-959.90   Min.   :-96.220   Min.   :-73.3000   Min.   :-982.95
 1st Qu.:   8.60   1st Qu.:  1.000   1st Qu.:  0.0000   1st Qu.:  15.50
 Median :  12.80   Median :  2.800   Median :  0.0000   Median :  20.30
 Mean   :  18.52   Mean   :  3.419   Mean   :  0.5339   Mean   :  27.27
 3rd Qu.:  20.50   3rd Qu.:  4.250   3rd Qu.:  0.0000   3rd Qu.:  29.04
 Max.   :2203.10   Max.   :984.300   Max.   :196.9900   Max.   :2208.10
```

```
tpep_pickup_datetime:  0 tpep_dropoff_datetime:  0 passenger_count:  236179 trip_distance:  0 PULocationID:  0 DOLocationID:  0 payment_type:  0 fare_amount:  0 tip_amount:  0 tolls_amount:  0 total_amount:  0
787.6 Mb
265  4
```

A data.table: 6 × 4

LocationID	Borough	Zone	service_zone
<int>	<chr>	<chr>	<chr>
1	EWR	Newark Airport	EWR
2	Queens	Jamaica Bay	Boro Zone
3	Bronx	Allerton/Pelham Gardens	Boro Zone
4	Manhattan	Alphabet City	Yellow Zone
5	Staten Island	Arden Heights	Boro Zone
6	Staten Island	Arrochar/Fort Wadsworth	Boro Zone

```r
# duplicates
n_dup <- sum(duplicated(trips))
cat("duplicate rows:", n_dup, "\n")
trips <- unique(trips)
```

```
duplicate rows: 0
```

```
# how many rows break each validity rule (one row can break more than one)
```

A data.table: 1 × 2

pickup_unmatched	dropoff_unmatched
<int>	<int>
0	0

A data.table: 6 × 18

passenger_count	trip_distance	PULocationID	DOLocationID	fare_amount	tip_amount	tolls_amount	total_amount	payment	duration_min	hour	day	month	wday	PU_borough	PU_zone	DO_borough	DO_zone
<int>	<dbl>	<int>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<fct>	<dbl>	<int>	<int>	<int>	<fct>	<chr>	<chr>	<chr>	<chr>
1	0.97	161	141	9.3	0.00	0	14.30	Cash	8.433333	0	1	1	Sun	Manhattan	Midtown Center	Manhattan	Lenox Hill West
1	1.10	43	237	7.9	4.00	0	16.90	Credit card	6.316667	0	1	1	Sun	Manhattan	Central Park	Manhattan	Upper East Side South
1	2.51	48	238	14.9	15.00	0	34.90	Credit card	12.750000	0	1	1	Sun	Manhattan	Clinton East	Manhattan	Upper West Side North
1	1.43	107	79	11.4	3.28	0	19.68	Credit card	10.833333	0	1	1	Sun	Manhattan	Gramercy	Manhattan	East Village
1	1.84	161	137	12.8	10.00	0	27.80	Credit card	12.300000	0	1	1	Sun	Manhattan	Midtown Center	Manhattan	Kips Bay
1	1.66	239	143	12.1	3.42	0	20.52	Credit card	10.450000	0	1	1	Sun	Manhattan	Upper West Side South	Manhattan	Lincoln Square West

fare_not_positive	82873
total_not_positive	81601
distance_zero_or_negative	135488
distance_over_100_miles	297
passengers_missing_or_odd	393040
duration_under_1_or_over_6h	113259
pickup_outside_jan_mar_2023	139

Conclusion

Task 2: Transportation data analytics

All of this is done with data.table grouping. Each result is stored in an object so it can be reused for plots later.

A data.table: 24 × 4

hour	trips	avg_fare	revenue
<int>	<int>	<dbl>	<dbl>
0	237075	19.63974	6803257
1	159180	17.77168	4178778
2	103572	16.69605	2570651
3	70431	17.74865	1836239
4	44783	22.84133	1453150
5	47389	27.00867	1785700
6	120231	22.61288	3763724
7	240795	18.95813	6492862
8	332463	17.55556	8425864
9	376708	17.86123	9737551
10	408146	17.90074	10548852
11	442383	17.85601	11397590
12	480514	18.25538	12628618
13	498673	18.99389	13556993
14	539429	19.98115	15366142
15	550751	19.66775	15420296
16	552382	19.68394	16569950
17	601296	18.59526	17292364
18	629363	17.19838	17058462
19	564208	17.57745	15551295
20	493323	18.02482	13388225
21	485919	18.41928	13414686
22	446336	19.27771	12729355
23	346615	20.32243	10322640

A data.table: 7 × 4

wday	trips	avg_fare	revenue
<fct>	<int>	<dbl>	<dbl>
Sun	1102412	19.80830	31471942
Mon	1067830	19.54416	30763420
Tue	1249607	18.40047	34292163
Wed	1335135	18.47833	36821447
Thu	1400226	18.73605	39062374
Fri	1369951	18.48235	37831329
Sat	1246804	17.67399	32050568

A data.table: 3 × 4

month	trips	avg_fare	revenue
<int>	<int>	<dbl>	<dbl>
1	2872830	18.50888	78506303
2	2721823	18.40972	74221873
3	3177312	19.09820	89565068

A data.table: 4 × 4

period	trips	avg_fare	revenue
<chr>	<int>	<dbl>	<dbl>
Evening (17-21)	2774109	17.93908	76705033
Afternoon (12-16)	2621749	19.34861	73541998
Morning (5-11)	1968115	18.46135	52152144
Night (22-4)	1407992	19.27254	39894069

```
# 5 and 6. routes (pickup zone to dropoff zone)
routes <- trips[!is.na(PU_zone) & !is.na(DO_zone),
                .(trips = .N, revenue = sum(total_amount), avg_fare = mean(fare_amount)),
                by = .(PU_zone, DO_zone)]
top_routes_count   <- routes[order(-trips)][1:10]
top_routes_revenue <- routes[order(-revenue)][1:10]
top_routes_count
top_routes_revenue
```

A data.table: 10 × 5

PU_zone	DO_zone	trips	revenue	avg_fare
<chr>	<chr>	<int>	<dbl>	<dbl>
Upper East Side South	Upper East Side North	61241	944574.3	8.643664
N/A	N/A	54955	1497974.8	19.096584
Upper East Side North	Upper East Side South	52080	836115.7	9.284810
Upper East Side North	Upper East Side North	39458	518492.7	6.714752
Upper East Side South	Upper East Side South	38494	524078.7	7.148571
Midtown Center	Upper East Side South	27372	461797.1	9.775037
Upper East Side South	Midtown Center	26713	440501.4	9.819474
Midtown Center	Upper East Side North	24897	536944.9	13.659678
Lincoln Square East	Upper West Side South	23531	351262.1	8.020938
Upper West Side South	Upper West Side North	23203	320884.8	7.177053

A data.table: 10 × 5

PU_zone	DO_zone	trips	revenue	avg_fare
<chr>	<chr>	<int>	<dbl>	<dbl>
JFK Airport	Outside of NYC	14307	1744899.7	103.066114
JFK Airport	Times Sq/Theatre District	18772	1734062.4	70.108553
N/A	N/A	54955	1497974.8	19.096584
LaGuardia Airport	Times Sq/Theatre District	16737	1244275.3	47.146606

```
# 8. payment method by pickup borough (share of trips inside each borough, in %)
pay_borough <- trips[, .N, by = .(PU_borough, payment)]
pay_borough[, share := round(100 * N / sum(N), 1), by = PU_borough]
dcast(pay_borough, PU_borough ~ payment, value.var = "share", fill = 0)
```

A data.table: 8 × 5

PU_borough	Credit card	Cash	No charge	Dispute
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Bronx	82.8	16.1	0.5	0.7
Brooklyn	80.3	18.4	0.6	0.8
EWR	70.1	22.4	6.0	1.5
Manhattan	82.7	16.5	0.3	0.5
N/A	68.8	26.1	2.1	3.0
Queens	77.2	21.6	0.4	0.7
Staten Island	95.2	4.4	0.2	0.2
Unknown	81.5	17.5	0.2	0.7

```
correlation: 0.963
  (Intercept) trip_distance
        6.013          3.712
```

A data.table: 8 × 4

dist_bin	trips	avg_fare	fare_per_mile
<fct>	<int>	<dbl>	<dbl>
(0,1]	1944531	7.36205	10.258325
(1,2]	2938010	11.32193	7.703123
(2,3]	1421634	16.02515	6.541275
(3,5]	1009955	21.80679	5.738155
(5,10]	737959	33.86876	4.662567
(10,20]	629289	61.40579	4.099698
(20,50]	89711	85.78050	3.695793
(50,100]	876	279.86668	4.632034

Task 3: Functional and efficient R programming

Two kinds of operations are compared.

Operation A is row by row: fare per mile for every trip. This is the case where apply and lapply feel natural but are slow, so I use a sample of 100,000 trips for it. Running apply on all millions of rows would take far too long.

Operation B is a group aggregation: average fare for each hour of the day, on a sample of 1 million trips, using functional (lapply and purrr over split groups), vectorized base R (tapply, rowsum), aggregate and data.table.

First I check that every method gives the same answer, then I time them.

```
set.seed(2)
n_mid <- min(nrow(trips), 1000000)
mid_dt <- trips[sample(.N, n_mid), .(hour, fare_amount, total_amount, PULocationID)]
mid_df <- as.data.frame(mid_dt)

grp_aggregate <- function(df) aggregate(fare_amount ~ hour, data = df, FUN = mean)$fare_amount
grp_lapply    <- function(df) unlist(lapply(split(df$fare_amount, df$hour), mean), use.names = FALSE)
grp_map       <- function(df) unname(purrr::map_dbl(split(df$fare_amount, df$hour), mean))
grp_tapply    <- function(df) as.vector(tapply(df$fare_amount, df$hour, mean))
grp_rowsum    <- function(df) as.vector(rowsum(df$fare_amount, df$hour)) / as.vector(table(df$hour))
grp_dt        <- function(dt) dt[, .(m = mean(fare_amount)), keyby = hour]$m

ref <- grp_dt(mid_dt)
c(aggregate = isTRUE(all.equal(grp_aggregate(mid_df), ref)),
  lapply    = isTRUE(all.equal(grp_lapply(mid_df), ref)),
  map       = isTRUE(all.equal(grp_map(mid_df), ref)),
  tapply    = isTRUE(all.equal(grp_tapply(mid_df), ref)),
  rowsum    = isTRUE(all.equal(grp_rowsum(mid_df), ref)))
```

```
aggregate:  TRUE lapply:  TRUE map:  TRUE tapply:  TRUE rowsum:  TRUE
```

```
mb_grp <- microbenchmark(
  "aggregate()"             = grp_aggregate(mid_df),
  "lapply(split())"         = grp_lapply(mid_df),
  "purrr::map_dbl(split())" = grp_map(mid_df),
  "tapply()"                = grp_tapply(mid_df),
  "rowsum()"                = grp_rowsum(mid_df),
  "data.table"              = grp_dt(mid_dt),
  times = 5)
print(summary(mb_grp, unit = "ms")[, c("expr", "min", "median", "max")])
```

```
                   expr       min    median       max
1           aggregate() 378.25691 534.47866 755.76980
2        lapply(split())  38.42668  87.09982 101.12667
3 purrr::map_dbl(split())  38.19764  69.89612  82.95303
4               tapply()  58.32311  94.21635 142.40657
5               rowsum()  60.10914  64.38750 147.77883
6            data.table  38.01769  44.79385  54.01412
```

Comparison of the approaches (Task 3)

Execution time and memory are in the tables above. The other criteria:

Approach	Code complexity	Readability	Suited for large data
apply() on rows	short	easy to follow but hides a slow loop	no, converts the data frame to a matrix and calls R code once per row
lapply() / purrr::map()	short	easy, purrr is neat for several inputs	only when each call does real work, because the per call overhead is paid every time
vectorized base R	shortest for element wise maths	very clear	yes for arithmetic, the loop runs in compiled code
tapply / rowsum	short	clear	good, but each call does one thing on one column
aggregate()	short formula	clear	slow and memory hungry on millions of rows
data.table	short, one syntax for filter, group and join	clear once DT[i, j, by] is learnt	yes, built for this

The reasons for the speed differences: apply, lapply and map call an R function again and again, and every call has overhead. Vectorized code and data.table push the whole loop into C. data.table also groups with radix ordering, uses several threads, computes the mean per group internally without calling R for each group, and changes columns by reference so it does not copy the table.

```
# A. sequential
t_seq <- system.time({
  res_seq <- foreach(d = part_list, .combine = rbind) %do% heavy_job(d, B_boot, n_samp)
})[["elapsed"]]
cat("sequential time:", round(t_seq, 2), "sec\n")
round(res_seq, 4)
```

```
sequential time: 51.47 sec
```

A matrix: 3 × 4 of type dbl

	month	slope	ci_low	ci_high
result.1	1	3.6890	3.6733	3.7040
result.2	2	3.7113	3.6951	3.7268
result.3	3	3.7176	3.7022	3.7328

```
# B. parallel with foreach + doParallel
n_workers <- min(detectCores(), length(part_list))
cat("workers used:", n_workers, "\n")

t_setup <- system.time({
  cl <- makeCluster(n_workers)
  registerDoParallel(cl)
})[["elapsed"]]

t_par <- system.time({
  res_par <- foreach(d = part_list, .combine = rbind, .export = c("heavy_job", "B_boot", "n_samp")) %dopar%
    heavy_job(d, B_boot, n_samp)
})[["elapsed"]]

cat("cluster start up:", round(t_setup, 2), "sec | parallel time:", round(t_par, 2), "sec\n")
cat("same results as sequential:", isTRUE(all.equal(res_seq, res_par, check.attributes = FALSE)), "\n")

speedup <- t_seq / t_par
cat("Speedup =", round(speedup, 2), "| efficiency =", round(100 * speedup / n_workers, 1), "% of the ideal\n")
```

```
workers used: 2
Warning message in e$fun(obj, substitute(ex), parent.frame(), e$data):
"already exporting variable(s): heavy_job, B_boot, n_samp"
```

Discussion of the parallel results

The speedup of the heavy job is printed above. The ideal speedup with k workers is k, but it is never reached. Reasons: the data of every partition has to be sent to the worker process, results are sent back, starting the cluster costs time, there are only 3 partitions so some workers can sit idle or one worker gets two months while another gets one, and Colab usually gives only 2 cores.

The light job shows the other side. When each task is tiny, the cost of sending data around is bigger than the work, so parallel can be slower than sequential. Parallel processing only helps when the work per task is large compared with the cost of moving the data.

Task 5: Performance benchmarking and optimization

All timings in one table. Median time in milliseconds, Relative is the median divided by the fastest approach in the same comparison, Peak_MB is the approximate extra memory from the earlier cell.

```
Warning message in e$fun(obj, substitute(ex), parent.frame(), e$data):
“already exporting variable(s): heavy_job, B_boot, n_samp”
```

A data.table: 2 × 4

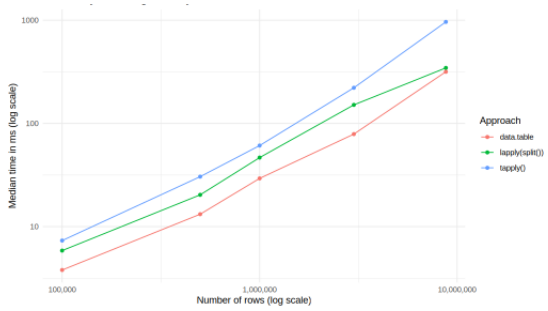
workers	time_sec	speedup	efficiency_pct
<dbl>	<dbl>	<dbl>	<dbl>
1	51.472	1.00	100.0
2	46.049	1.12	55.9

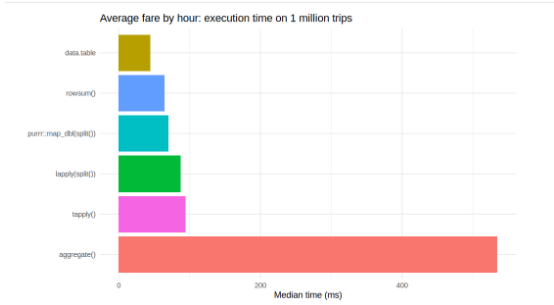
	Comparison	Approach	Median_ms	Peak_MB	Relative
	<char>	<char>	<num>	<num>	<num>
1:	Row-wise fare per mile (100k rows)	apply()	776.954	119.9	3392.8
2:	Row-wise fare per mile (100k rows)	lapply()	485.182	111.8	2118.7
3:	Row-wise fare per mile (100k rows)	purrr::map2_dbl()	165.060	83.1	720.8
4:	Row-wise fare per mile (100k rows)	vectorized	0.229	0.8	1.0
5:	Row-wise fare per mile (100k rows)	data.table	1.315	1.0	5.7
6:	Average fare by hour (1M rows)	aggregate()	534.479	189.5	11.9
7:	Average fare by hour (1M rows)	lapply(split())	87.100	27.2	1.9
8:	Average fare by hour (1M rows)	purrr::map_dbl(split())	69.896	27.2	1.6
9:	Average fare by hour (1M rows)	tapply()	94.216	34.8	2.1
10:	Average fare by hour (1M rows)	rowsum()	64.387	65.7	1.4
11:	Average fare by hour (1M rows)	data.table	44.794	28.3	1.0
12:	Join with zone table (1M rows)	merge() base R	1971.627	391.5	27.4
13:	Join with zone table (1M rows)	data.table join	71.872	57.5	1.0
14:	Bootstrap job on 3 month partitions	Sequential (foreach %do%)	51472.000	NA	1.1
15:	Bootstrap job on 3 month partitions	Parallel (foreach %dopar%, 2 workers)	46984.000	NA	1.0
16:	Light job on 3 month partitions	Sequential (foreach %do%)	40.000	NA	1.0
17:	Light job on 3 month partitions	Parallel (foreach %dopar%, 2 workers)	826.000	NA	20.6

A data.table: 15 × 3

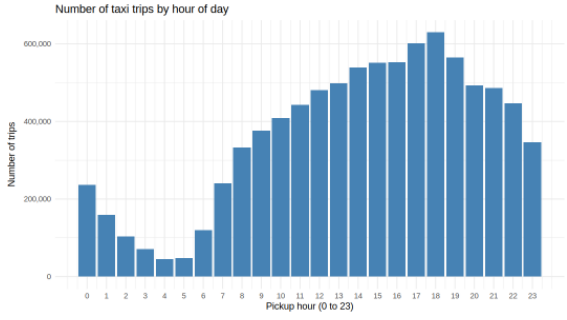
rows	approach	median_ms
<dbl>	<chr>	<dbl>
100000	tapply()	7.323388
100000	lapply(split())	5.856640
100000	data.table	3.797439
500000	tapply()	30.500852
500000	lapply(split())	20.320606
500000	data.table	13.191809
1000000	tapply()	61.030340
1000000	lapply(split())	46.653609
1000000	data.table	29.328467
3000000	tapply()	221.455510
3000000	lapply(split())	151.256625
3000000	data.table	78.803881
8771965	tapply()	963.443557
8771965	lapply(split())	345.331832
8771965	data.table	316.783832

Scalability of average fare by hour

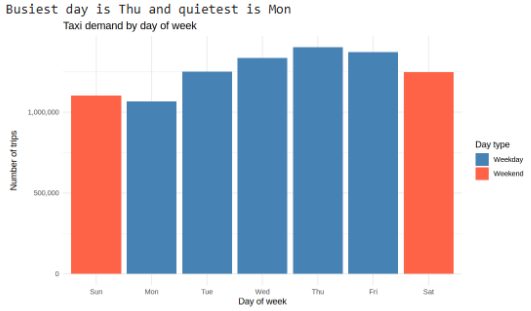




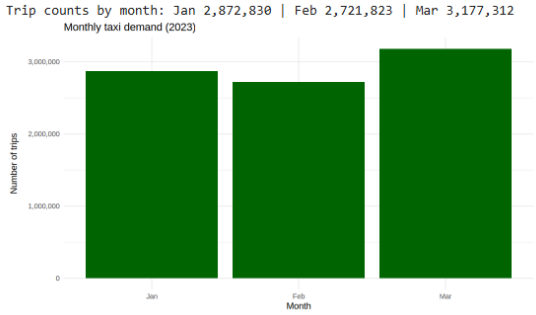
Demand is lowest at hour 4 and peaks at hour 18 with 629,363 trips.



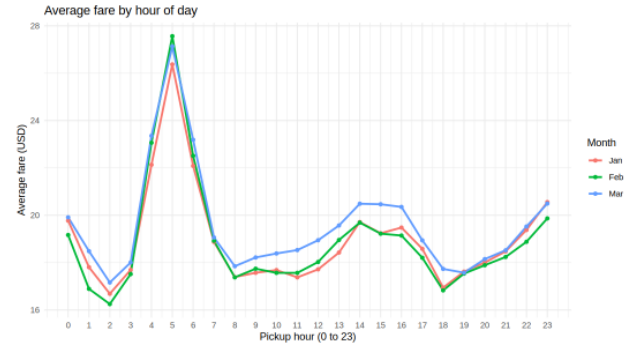
```
Busiest day is Thu and quietest is Mon
```



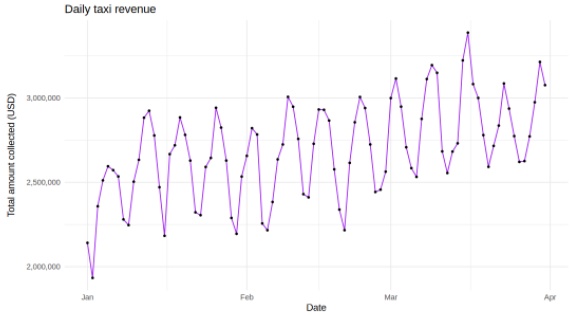

```
Trip counts by month: Jan 2,872,830 | Feb 2,721,823 | Mar 3,177,312
```

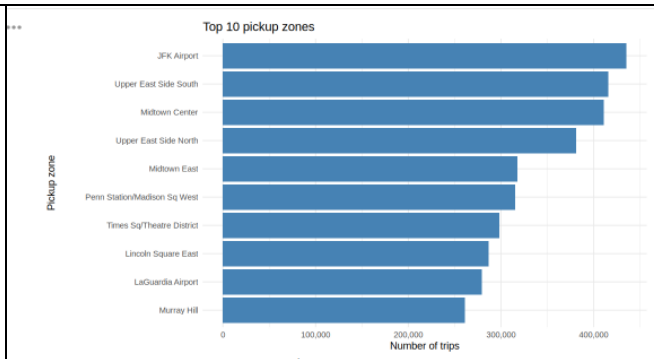



```
Average fare is highest at hour 5 ( 27.01 USD ) and lowest at hour 2 ( 16.7 USD ).
```

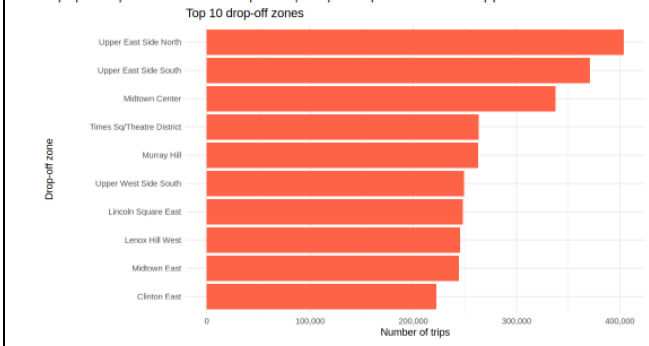


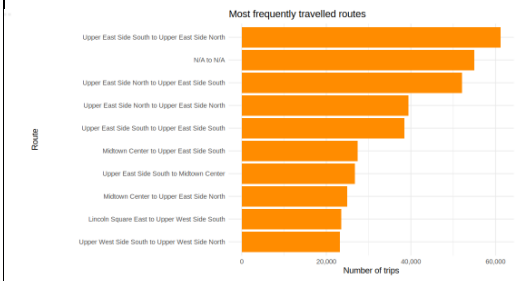

```
Best revenue day: 2023-03-16 with 3,386,968 USD; weakest day: 2023-01-02 with 1,933,796 USD
```



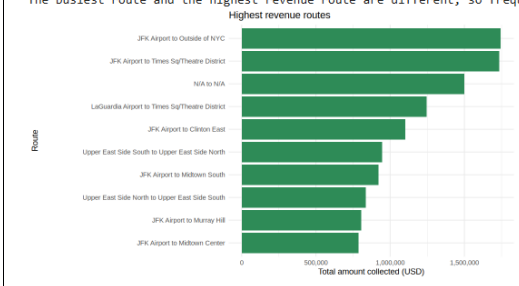


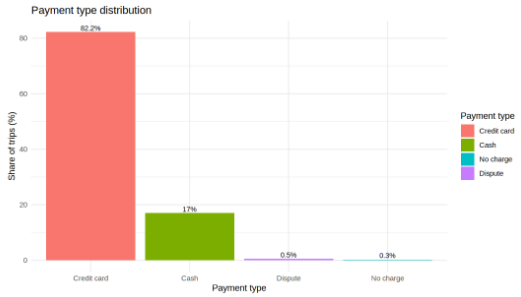

```
Top pickup zone: JFK Airport | top drop-off zone: Upper East Side North
```



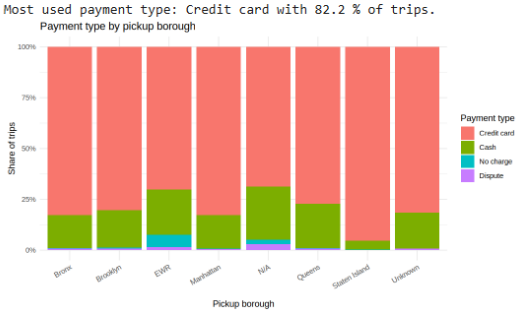


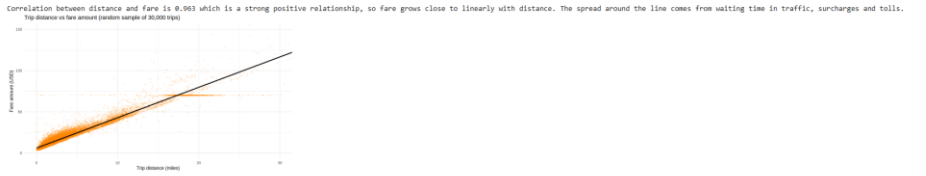

```
The busiest route and the highest revenue route are different, so frequent does not always mean most profitable.
```





Most used payment type: Credit card with 82.2 % of trips.





Task 7: Critical analysis and recommendation

1. Fastest approach in each comparison:

Comparison	Fastest
<char>	<char>
1: Row-wise fare per mile (100k rows)	vectorized
2: Average fare by hour (1M rows)	data.table
3: Join with zone table (1M rows)	data.table join
4: Bootstrap job on 3 month partitions	Parallel (foreach %dopar%, 2 workers)
5: Light job on 3 month partitions	Sequential (foreach %do%)

Median_ms
<num>
1: 0.229
2: 44.794

	3: 71.872 4: 46984.000 5: 40.000 Q2. Group average on 1M rows: aggregate() took 534.479 ms, tapply() took 94.216 ms, data.table took 44.794 ms. Join on 1M rows: merge() took 1971.627 ms, data.table took 71.872 ms. Q3 and Q4. Row-wise fare per mile on 100k rows: apply() 776.954 ms, lapply() 485.182 ms, purrr::map2_dbl() 165.06 ms, vectorized 0.229 ms, data.table 1.315 ms. Q5. Heavy job: sequential 51.47 sec, parallel with 2 workers 46.98 sec, speedup 1.1 -> parallel gave little or no gain on this job. Q6. Light job: sequential 0.04 sec, parallel 0.826 sec, speedup 0.05 -> parallel was slower here, so parallel is not always faster. The overhead of starting workers and sending data was bigger than the work. **Answers** **2. How does data.table improve the processing of large datasets?** It does the grouping, joining and sorting in compiled C code with radix ordering and several threads, it computes common summaries like mean and sum per group internally, it adds or changes columns by reference so the table is not copied, and one compact syntax (DT[i, j, by]) covers filter, select, group and join. Less copying also means less memory. **3. When is functional programming preferable?** When there is no ready made vectorized function and each element needs its own custom logic, when looping over files, models or partitions (like the month partitions above), and when the code reads better as map over a list.

	The cost is a function call per element, so it should not be used for simple arithmetic on millions of rows. **4. When do vectorized operations provide better performance?** For element wise calculations on whole columns (fare divided by distance, unit conversions, comparisons). The loop runs in compiled code, so it is much faster than apply or lapply and uses little extra memory. **7. Trade-offs.** apply and lapply are easy to write but slow and sometimes memory hungry. Vectorized code is fast and simple but only fits element wise maths. data.table is fast and memory efficient but has its own syntax that has to be learnt, and update by reference can surprise people used to copy on modify. Parallel code can cut the time of heavy independent jobs but adds setup code, uses more total memory because data is copied to workers, and can be slower for small jobs. **8. Recommendation.** For large scale analytics in R I would use data.table for loading, cleaning, joining and aggregating, vectorized expressions inside it for column calculations, purrr or lapply only for looping over big independent units like partitions, and foreach with doParallel only for heavy independent jobs where the per task time is large compared with the cost of moving data. The verdict cell below checks this against my own numbers.
Conclusion	Row by row functional code is the slowest, vectorized code is best for element wise work, and data.table is the best all round tool for filtering, joining and grouping. Parallel processing helped only when each partition had a lot of computation to do, and it did not reach the ideal speedup because of the cost of starting workers and moving data, the small number of partitions and the limited number of cores. The workflow of cleaning, joining, analysing, benchmarking and visualising is one pipeline that scales to larger months of data without changing the code.